# PROJECT REPORT

## TLB Address Translation Simulator: An Interactive Approach to Virtual-to-Physical Translation and TLB Performance Analysis


Submitted by:

P V Sai Likith (24BCI0174)

M V S Sujan (24BCE2294)

Shrijit (24BCE2469)

Rajnihil (24BCT0191)

# CONTENTS

1. Problem Statement ..... 3
2. Objectives ..... 4
3. System Design ..... 4
4. Algorithm and Pseudocode ..... 8
5. Implementation ..... 10
6. Screenshots and Results ..... 13
7. Testing and Validation ..... 18
8. Conclusion and Future Enhancements ..... 21
9. References ..... 21

## 1. Problem Statement

Every program runs in its own **virtual address space**, but main memory understands only **physical addresses**. With **paging**, a virtual address (VA) is split into a **virtual page number (VPN)** and an **offset**. A **page table** stored in main memory maps each VPN to a **physical frame number (PFN)**, and the physical address is PA = PFN × page size + offset.

Reading the page table on every memory reference would make each access cost at least two memory trips, and $k + 1$ trips for a $k$-level page table. Processors therefore use a **Translation Lookaside Buffer (TLB)**, a small, fast cache of recent translations. On a **TLB hit** the PFN is available in about 10 ns; on a **TLB miss** the page table must be walked; and if the page is not in memory, a **page fault** hands control to the operating system.

All of this happens inside the hardware in nanoseconds, so it cannot be observed. Textbooks show only static diagrams, and hand-solving translation problems is slow and error-prone. Students therefore find it hard to see why a hit or a miss happens, how the replacement policy chooses a victim, or how much the TLB actually speeds up memory access.

**Problem statement:** *To design and implement a simulator that models virtual-to-physical address translation using a TLB and a page table, shows every step of each translation (address split, TLB lookup, page-table walk, page fault, protection check and physical-address formation), and measures how the TLB size, organisation and replacement policy affect the hit ratio and the effective access time (EAT).*

**Scope.** The simulator supports 4- to 32-bit virtual and physical addresses, power-of-two page sizes, 1 to 4 page-table levels, TLBs of 1 to 64 entries (fully associative, set-associative or direct-mapped), the replacement policies FIFO, LRU, LFU, CLOCK, RANDOM and OPT, page faults in report-only or demand-paging mode, read/write protection, and up to four processes. It is a teaching model and is not cycle-accurate.

## 2. Objectives

The project has five objectives:

1. **Translate addresses step by step.** Split any virtual address into VPN and offset (and set index, tag and page-table level indices), and compute the physical address, showing each step at bit level.
2. **Simulate the TLB and page table.** Model TLB hits, misses, fills and evictions with six replacement policies (FIFO, LRU, LFU, CLOCK, RANDOM and OPT), for fully associative, set-associative and direct-mapped TLBs, together with page-table walks of 1 to 4 levels.
3. **Handle page faults and protection.** Detect page faults (report-only or demand paging with frame replacement and dirty write-back) and write-protection faults.
4. **Measure performance.** Compute the hit ratio, misses, faults and evictions, and compare the measured effective access time with the textbook EAT formula and with a system that has no TLB.
5. **Compare policies and verify correctness.** Run the same trace under every policy and TLB size, detect Belady's anomaly automatically, and validate every result with automated tests derived from textbook examples.

## 3. System Design

The design is described with an architecture diagram, a flowchart and three UML diagrams (use-case, class and sequence).

### 3.1 System architecture

The system has a layered architecture (Figure 1). The user works with ten web pages. A shared interface shell provides navigation, and a session layer keeps the history of operations and replays it through a deterministic simulation engine that contains all of the memory-management logic. The same engine logic is also implemented in Python as a command-line program (Section 5).

- **Presentation layer:** ten pages, each with one job: Home, Machine, Translate, Timeline, Structures, Analytics, Policy lab, Learn, Self-test and Report.
- **Interface shell:** navigation tabs, the journey stepper, the command palette, animations, the calculation player and the printable report.
- **Session layer:** stores the configuration, the list of operations and the time cursor, and replays them through the engine. This gives rewind, branching and "what-if" replay.
- **Simulation engine:** address decomposition, TLB lookup and replacement, page-table walk, fault handling, protection, statistics and EAT, policy comparison and invariant checks.

- **Persistence and verification:** browser localStorage, share links and CSV/JSON export; 96 automated engine tests and 29 page tests.

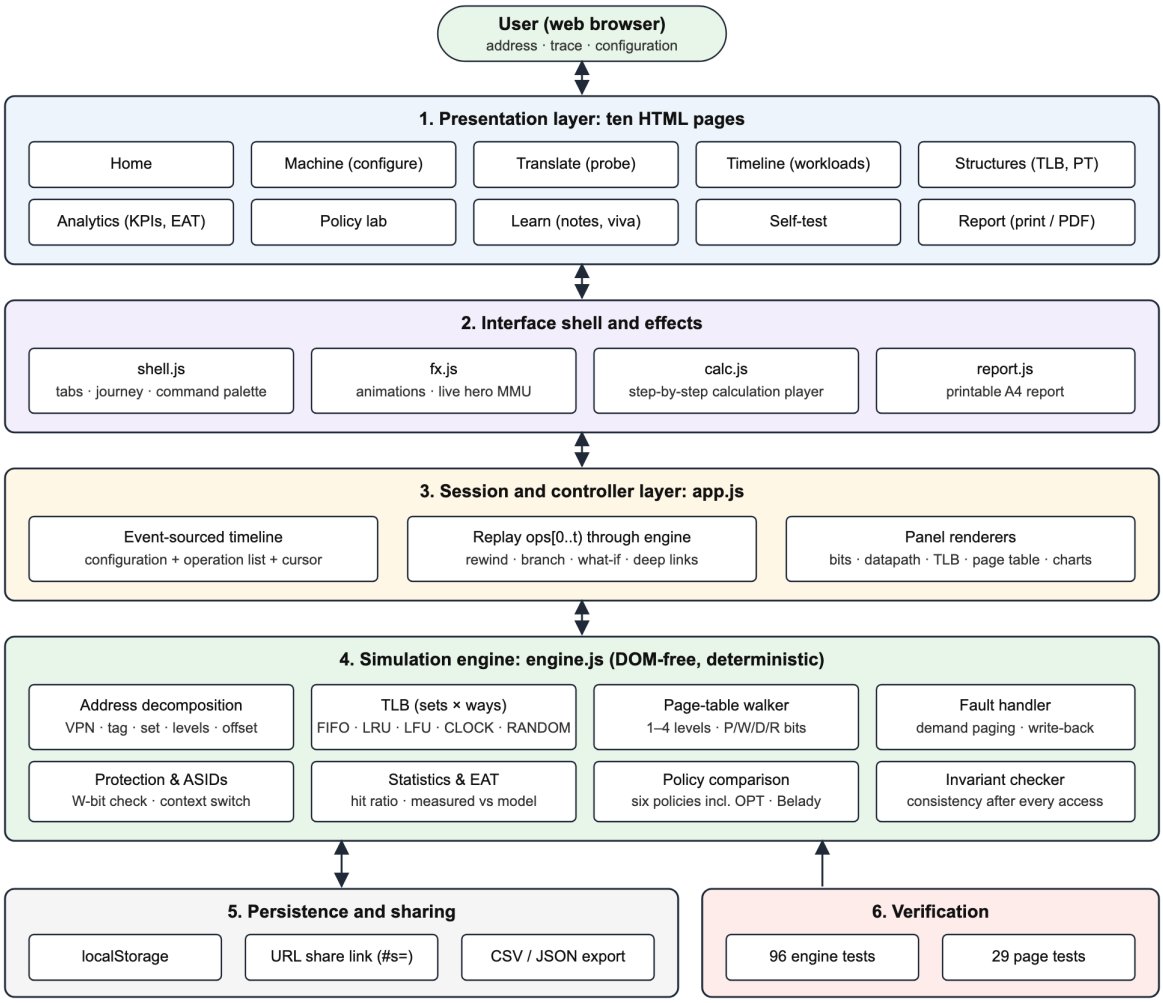



**Figure 1:** *System architecture*

## 3.2 Flowchart

Figure 2 shows how one memory access is processed.

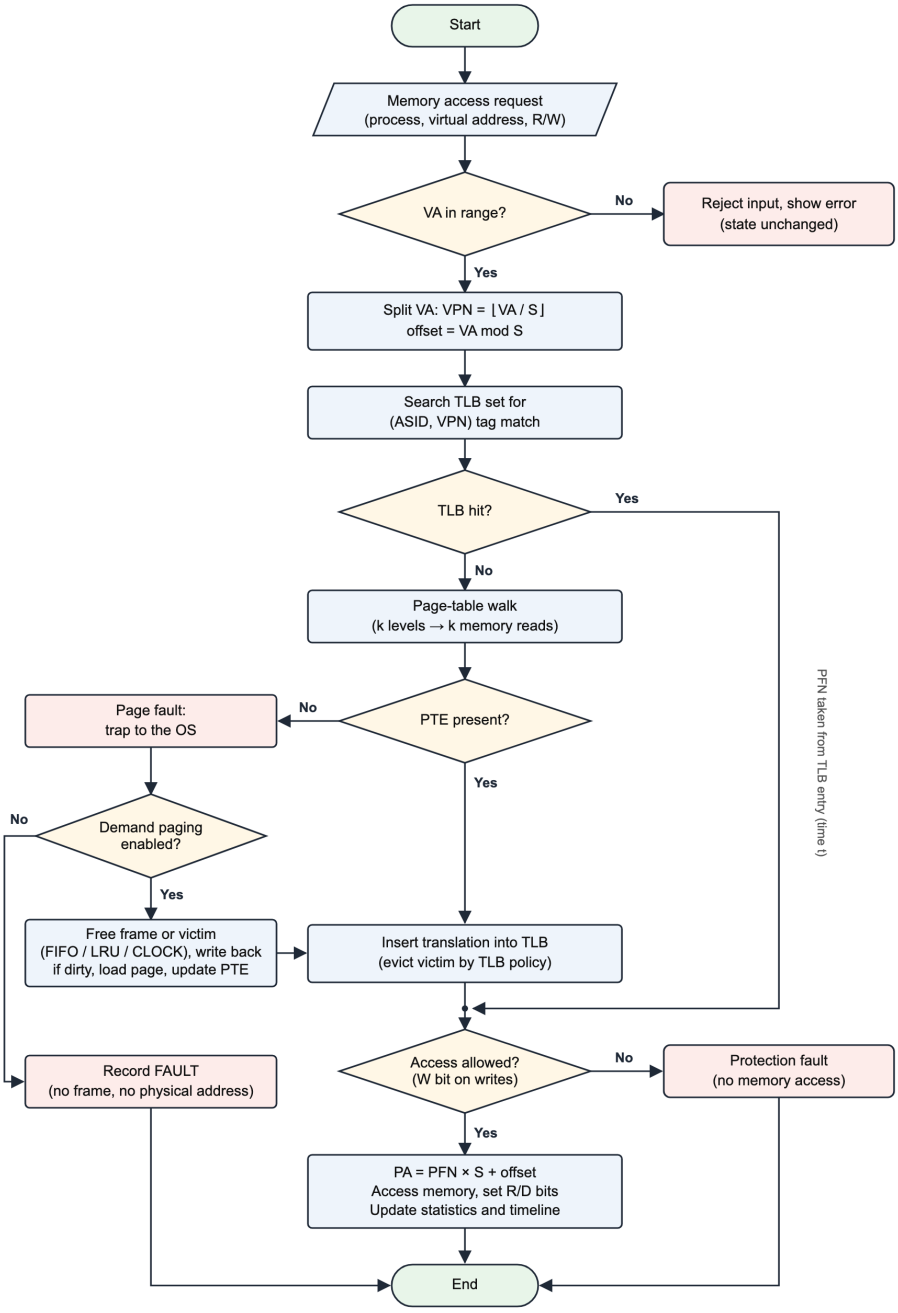



**Figure 2:** *Flowchart of one memory access*

### 3.3 UML use-case diagram

Figure 3 shows what the two kinds of user can do with the system. Students configure the machine, translate addresses, run traces and study the results; teachers and examiners use scenarios, policy comparison, the self-test and the report.

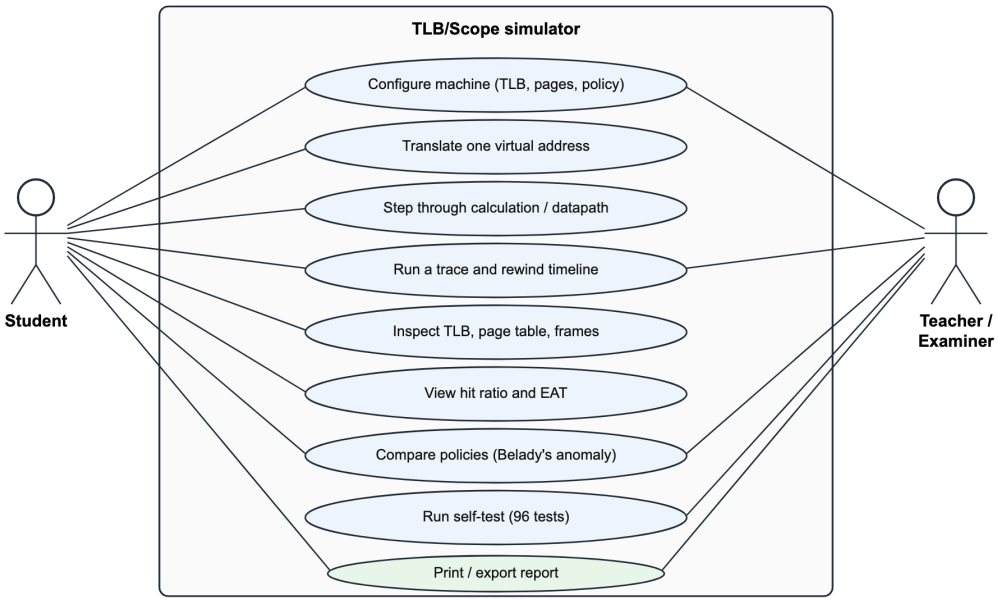



**Figure 3:** *UML use-case diagram*

### 3.4 UML class diagram

Figure 4 shows the classes of the Python implementation. **MMU** owns the TLB (a list of sets, each holding **TLBEntry** objects), the page table (a list of **PTE** objects) and the log of **Record** objects. **Config** holds the machine parameters and derives the number of pages, frames and TLB sets.

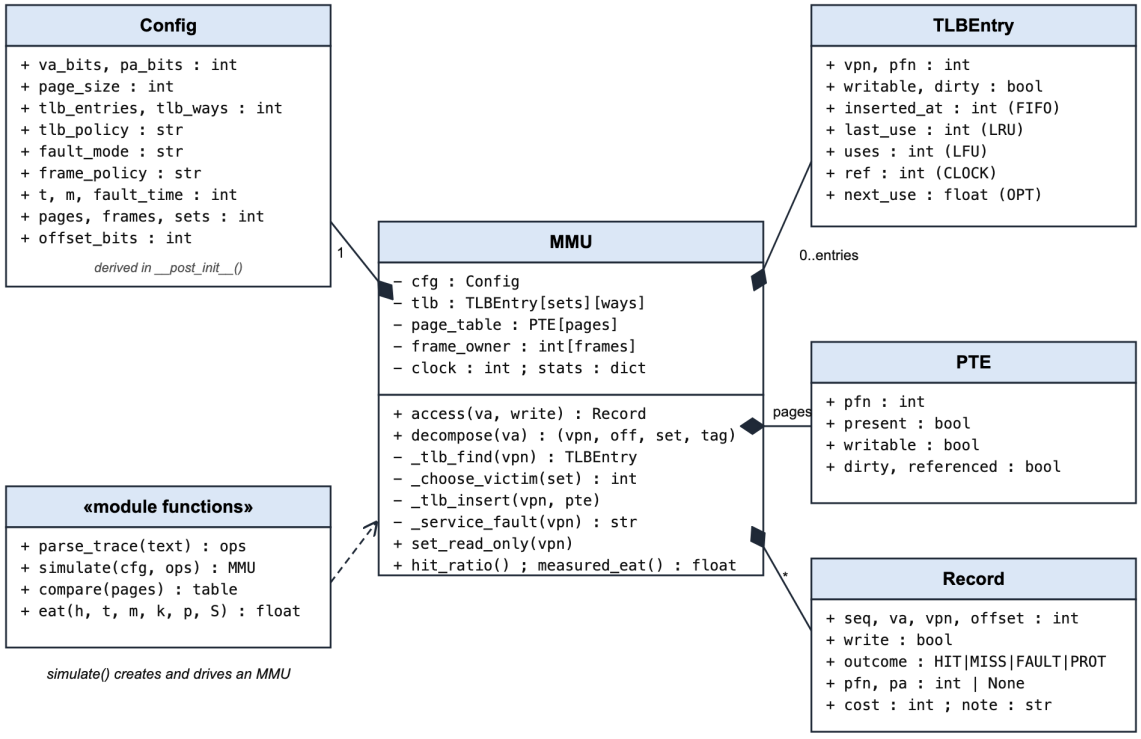



**Figure 4:** *UML class diagram (Python implementation)*

### 3.5 UML sequence diagram

Figure 5 shows the messages exchanged when a student translates the address 300. The alternative frame shows the hit path and the miss path.

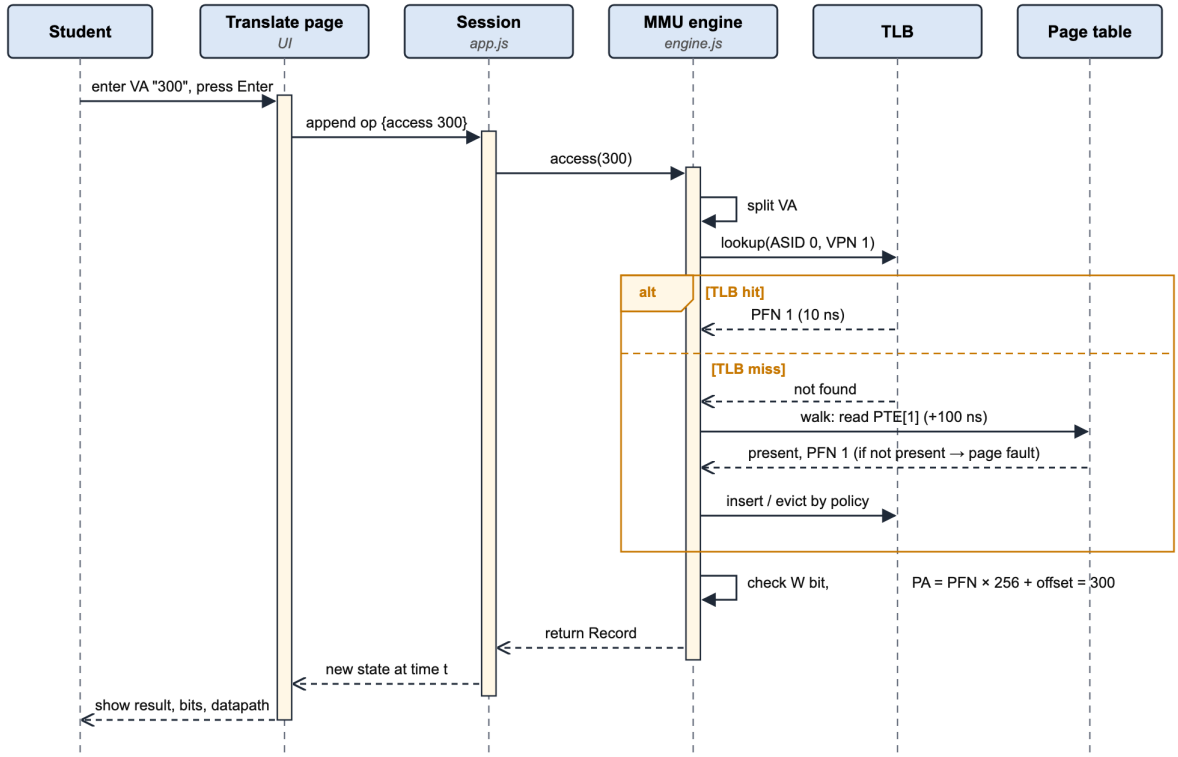



**Figure 5:** *UML sequence diagram for translating one address*

### 3.6 Key design decisions

- **Event-sourced timeline:** every user action is stored as an operation, and the state at time $t$ is obtained by replaying operations 0 to $t$. Because the engine is deterministic (RANDOM uses a fixed seed), this gives time travel, branching and what-if replay from one mechanism.
- **Separate engine:** the engine has no user-interface code, so it can be tested on its own in Node.js and reimplemented in Python.
- **Compact data structures:** the page table uses typed arrays (one PFN and one flag byte per page), so even a 32-bit address space with 4 KiB pages (2^20 entries) stays fast. Each frame records its owner page, which acts as an inverted page table for fast eviction.

## 4. Algorithm and Pseudocode

### 4.1 Formulas

Let $S$ be the page size, $t$ the TLB lookup time, $m$ the memory access time, $k$ the number of page-table levels, $h$ the hit ratio, $p$ the page-fault rate and $F$ the fault service time.

**Table 1:** *Formulas used by the simulator*

| Quantity | Formula |
|---|---|
| Offset bits | log2(S) |
| Virtual page number / offset | VPN = floor(VA / S); offset = VA mod S |
| TLB sets, set index and tag | sets = entries / ways; set = VPN mod sets; tag = floor(VPN / sets) |
| Physical address | PA = PFN × S + offset |
| Cost of a TLB hit / miss | t + m / t + k·m + m |
| Effective access time | EAT = h(t + m) + (1 − h)(t + (k + 1)m) + p·F |

## 4.2 Pseudocode: one memory access

### Pseudocode 1: ACCESS

```
ALGORITHM ACCESS(VA, write)
  INPUT : virtual address VA, flag write (true for a store)
  OUTPUT: outcome (HIT, MISS, FAULT or PROT), physical address PA, cost

  1  if VA < 0 or VA > 2^vaBits - 1 then
  2      reject "address out of range"          // state is not changed
  3  clock <- clock + 1
  4  VPN <- floor(VA / S);  offset <- VA mod S;  set <- VPN mod sets
  5  cost <- t                                  // TLB lookup
  6  entry <- search TLB[set] for VPN
  7  if entry found then                        // ---- TLB HIT
  8      update entry.lastUse, entry.uses, entry.ref
  9      outcome <- HIT
 10  else                                       // ---- TLB MISS
 11      cost <- cost + k * m                   // page-table walk
 12      if PageTable[VPN].present = false then  // ---- PAGE FAULT
 13          if faultMode = REPORT then
 14              return (FAULT, no PA, cost)
 15          frame <- free frame, else victim by FIFO / LRU
 16          if victim is dirty then write it back to disk
 17          invalidate victim PTE and its TLB entry
 18          load page into frame; update PageTable[VPN]
 19          cost <- cost + F
 20      way <- empty way in TLB[set], else CHOOSE_VICTIM(set)
 21      TLB[set][way] <- (VPN, PageTable[VPN].pfn, bits, clock)
 22      outcome <- MISS (or FAULT if the fault was serviced)
 23  if write and entry is read-only then
 24      return (PROT, no PA, cost)             // protection fault
 25  if write then mark page and entry dirty
 26  PA <- entry.pfn * S + offset
 27  cost <- cost + m                           // data access
 28  record (outcome, PA, cost) and update statistics
 29  return (outcome, PA, cost)
```

## 4.3 Pseudocode: choosing a TLB victim

### Pseudocode 2: CHOOSE_VICTIM

```
ALGORITHM CHOOSE_VICTIM(set)
  row <- TLB[set]
  if row has an empty way then return that way
  switch policy
    FIFO  : return the way with the smallest insertedAt
    LRU   : return the way with the smallest lastUse
    LFU   : return the way with the smallest uses (ties -> smallest lastUse)
    OPT   : return the way whose next use is farthest in the future
    RANDOM: return a seeded pseudo-random way
    CLOCK : loop
              if row[hand].ref = 1 then row[hand].ref <- 0; hand <- hand + 1
              else victim <- hand; hand <- hand + 1; return victim
```

### 4.4 Replacement policies

**Table 2:** *TLB replacement policies*

| Policy | Victim | Property |
|---|---|---|
| FIFO | Entry inserted earliest | Simple; can show Belady's anomaly |
| LRU | Entry used least recently | Stack algorithm; no Belady's anomaly |
| LFU | Entry used least often | Good for skewed workloads; slow to adapt |
| CLOCK | First entry with reference bit 0 | Cheap approximation of LRU (second chance) |
| RANDOM | Seeded random entry | No bookkeeping; reproducible because of the seed |
| OPT | Entry used farthest in the future | Optimal lower bound; needs the future trace |

### 4.5 Worked example

Default machine: 16-bit VA, 12-bit PA, 256-byte pages (256 pages, 16 frames, 8 offset bits), 4-entry fully associative LRU TLB, t = 10 ns, m = 100 ns.

- **VA 300:** VPN = floor(300 / 256) = 1, offset = 300 mod 256 = 44. The first access is a miss: the page table gives PFN 1, so PA = 1 × 256 + 44 = **300**, cost = 10 + 100 + 100 = **210 ns**.
- **VA 300 again:** the translation is now in the TLB, so it is a hit costing 10 + 100 = **110 ns**.
- **VA 0x1A2B (6699):** VPN = 26, offset = 43. Only pages 0 to 15 are in memory, so this is a **page fault**.
- **EAT** with h = 0.8: 0.8 × 110 + 0.2 × 210 = **130 ns**, compared with **200 ns** without a TLB.

## 5. Implementation

### 5.1 Technologies

The simulator was implemented twice with the same logic:

- **Python implementation** (`tlb_sim.py`, Python 3): a command-line simulator of the MMU, with a unit-test suite (`test_tlb_sim.py`). It needs only the Python standard library.
- **Interactive web implementation** (TLB/Scope): HTML5, CSS3 and JavaScript, with no frameworks and no back end. It provides the visual interface: bit-level view, datapath, timeline, structures, analytics, policy lab and report.

Both implementations produce identical results on the same input (Section 7.4).

**Table 3:** *Project structure*

| File / folder | Purpose |
|---|---|
| `python-implementation/tlb_sim.py` | Python MMU simulator: Config, TLBEntry, PTE, Record, MMU, simulate(), compare(), eat() |
| `python-implementation/test_tlb_sim.py` | 16 Python unit tests |

| File / folder | Purpose |
|---|---|
| tlb-simulator/js/engine.js | JavaScript MMU engine used by the web pages (721 lines) |
| tlb-simulator/js/app.js | Session, replay, persistence and panel renderers (1,406 lines) |
| tlb-simulator/js/shell.js, fx.js, calc.js, report.js | Navigation, animations, calculation player, printable report |
| tlb-simulator/*.html | Ten pages: index, machine, translate, timeline, structures, analytics, lab, learn, verify, report |
| tlb-simulator/tests/ | 96 engine tests and 29 page-level smoke tests |

## 5.2 Core Python code

The following listings are taken directly from `tlb_sim.py`. Listing 1 is the main translation routine and Listing 2 chooses which TLB entry to replace.

**Listing 1: MMU.access() — one memory access**

```
    def access(self, va, write=False, next_use=float("inf")):
        cfg, st = self.cfg, self.stats
        if not isinstance(va, int) or not 0 <= va <= cfg.va_max:
            raise ValueError(f"address {va} is outside the {cfg.va_bits}-bit "
                             f"virtual space (0-{cfg.va_max})")
        self.clock += 1
        st["accesses"] += 1
        st["writes" if write else "reads"] += 1
        vpn, offset, _, _ = self.decompose(va)
        pte = self.page_table[vpn]
        cost, note, fault = cfg.t, "", False

        entry = self._tlb_find(vpn)
        if entry:                                       # TLB HIT
            st["hits"] += 1
            entry.last_use, entry.ref, entry.next_use = self.clock, 1, next_use
            entry.uses += 1
            outcome = "HIT"
        else:                                           # TLB MISS -> page walk
            st["misses"] += 1
            cost += cfg.m
            if not pte.present:                         # PAGE FAULT
                st["page_faults"] += 1
                fault = True
                if cfg.fault_mode == "report":
                    return self._log(va, write, vpn, offset, "FAULT", None, None,
                                     cost, "not serviced")
                note = self._service_fault(vpn)
                pte = self.page_table[vpn]
                cost += cfg.fault_time
            entry, old = self._tlb_insert(vpn, pte, next_use)
            if old:
                note = f"TLB evict page {old.vpn}"
            outcome = "FAULT" if fault else "MISS"

        pte.referenced = True
        self.frame_used[entry.pfn] = self.clock
        if write and not entry.writable:                # PROTECTION FAULT
            st["prot_faults"] += 1
            return self._log(va, write, vpn, offset, "PROT", entry.pfn, None,
                             cost, "write to read-only page")
        if write:
            pte.dirty = entry.dirty = True
        cost += cfg.m                                   # data access
        pa = entry.pfn * cfg.page_size + offset
        return self._log(va, write, vpn, offset, outcome, entry.pfn, pa, cost, note)
```

**Listing 2: MMU._choose_victim() — TLB replacement policies**

```
    def _choose_victim(self, s):
        row = self.tlb[s]
        if None in row:
            return row.index(None)
        policy = self.cfg.tlb_policy
        if policy == "CLOCK":                       # second chance
            while True:
                h = self.clock_hand[s]
                if row[h].ref:
                    row[h].ref = 0
                    self.clock_hand[s] = (h + 1) % len(row)
                else:
                    self.clock_hand[s] = (h + 1) % len(row)
                    return h
        if policy == "RANDOM":
            return self.rng.randrange(len(row))
        key = {
            "FIFO": lambda e: (e.inserted_at,),
            "LRU": lambda e: (e.last_use,),
            "LFU": lambda e: (e.uses, e.last_use),
            "OPT": lambda e: (-e.next_use, e.inserted_at),
        }[policy]
        return min(range(len(row)), key=lambda w: key(row[w]))
```

**Listing 3: effective access time**

```
def eat(h, t=10, m=100, levels=1, p=0.0, fault_time=0):
    """Effective access time: h(t+m) + (1-h)(t+(k+1)m) + p*S."""
    return h * (t + m) + (1 - h) * (t + (levels + 1) * m) + p * fault_time
```

## 5.3 How to run

**Table 4:** *Commands*

| Task | Command |
|---|---|
| Run the Python demo | `python3 tlb_sim.py` |
| Run your own trace | `python3 tlb_sim.py --trace "300 300 0x1A2B 300:W"` |
| Change policy and TLB size | `python3 tlb_sim.py --trace "..." --policy FIFO --entries 3` |
| Compare all policies | `python3 tlb_sim.py --compare "1 2 3 4 1 2 5 1 2 3 4 5"` |
| Run the Python tests | `python3 -m unittest -v test_tlb_sim` |
| Start the web app | `python3 -m http.server 5173 --directory tlb-simulator`, then open http://localhost:5173 |
| Run the JavaScript tests | `npm test` (inside tlb-simulator) |

## 5.4 Web application pages

**Table 5:** *Pages of the web application*

| Step | Page | What it does |
|---|---|---|
| 1 | Machine | Configure address widths, page size, TLB size, organisation, policy, faults, processes and timing; ten one-click scenarios |
| 2 | Translate | Translate one address; show the result, bit-level view, MMU datapath and animated calculation |
| 3 | Timeline | Run a trace; logic-analyzer view; rewind, branch and replay; event table |
| 4 | Structures | TLB sets and ways, page-table map, physical frames |

| Step | Page | What it does |
|---|---|---|
| 5 | Analytics | Hit ratio, misses, faults, evictions, hit-ratio chart, EAT calculator |
| 6 | Policy lab | Six policies × seven TLB sizes; Belady's anomaly detection |
| 7 | Learn | Concept notes, viva questions, real-world mapping |
| 8 | Self-test | Runs the 96 engine tests in the browser |
| 9 | Report | Printable A4 report of the session |

# 6. Screenshots and Results

## 6.1 Python implementation output

Figure 6 shows the output of `python3 tlb_sim.py`. It runs the sample trace `300 300 301 1200 0x1A2B 0 256 512 768 0 300:W` on the default machine, then compares all policies on Belady's reference string.

```
Terminal — python3 tlb_sim.py

$ python3 tlb_sim.py
Machine: 16-bit VA, 12-bit PA, 256 B pages -> 256 pages, 16 frames, offset 8 bits
TLB    : 4 entries (1 set x 4 ways), LRU, faults=report, t=10 ns, m=100 ns

  T R/W     VA  VPN  OFF OUTCOME  PFN     PA  COST  NOTES
  1   R    300    1   44    MISS    1    300   210
  2   R    300    1   44     HIT    1    300   110
  3   R    301    1   45     HIT    1    301   110
  4   R   1200    4  176    MISS    4   1200   210
  5   R   6699   26   43   FAULT    -      -   110  not serviced
  6   R      0    0    0    MISS    0      0   210
  7   R    256    1    0     HIT    1    256   110
  8   R    512    2    0    MISS    2    512   210
  9   R    768    3    0    MISS    3    768   210  TLB evict page 4
 10   R      0    0    0     HIT    0      0   110
 11   W    300    1   44     HIT    1    300   110

Accesses 11 (10 R, 1 W) | hits 5 | misses 6 | page faults 1 | protection faults 0 | TLB evictions 1
Hit ratio      : 45.5 %
Measured EAT   : 155.45 ns
Model EAT (h=0.8): 130 ns   |   no TLB: 200 ns

--- Policy comparison on Belady's string 1 2 3 4 1 2 5 1 2 3 4 5 ---
Hit % by policy and TLB size
Policy        1      2      3      4      8
FIFO        0.0    0.0   25.0   16.7   58.3
LRU         0.0    0.0   16.7   33.3   58.3
LFU         0.0    0.0   16.7   33.3   58.3
CLOCK       0.0    0.0   25.0   16.7   58.3
OPT         0.0   25.0   41.7   50.0   58.3
Belady's anomaly: FIFO got worse going from 3 to 4 entries
Belady's anomaly: CLOCK got worse going from 3 to 4 entries
```

**Figure 6:** *Output of the Python simulator*

## 6.2 Web application

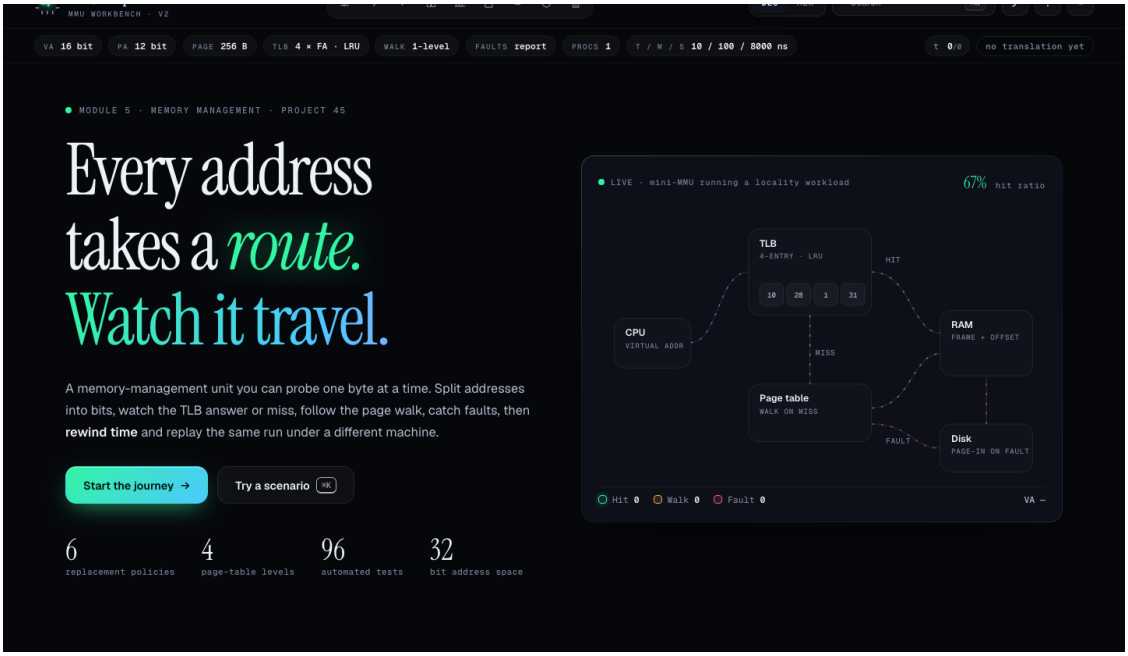



**Figure 7:** *Home page with the live mini-MMU*

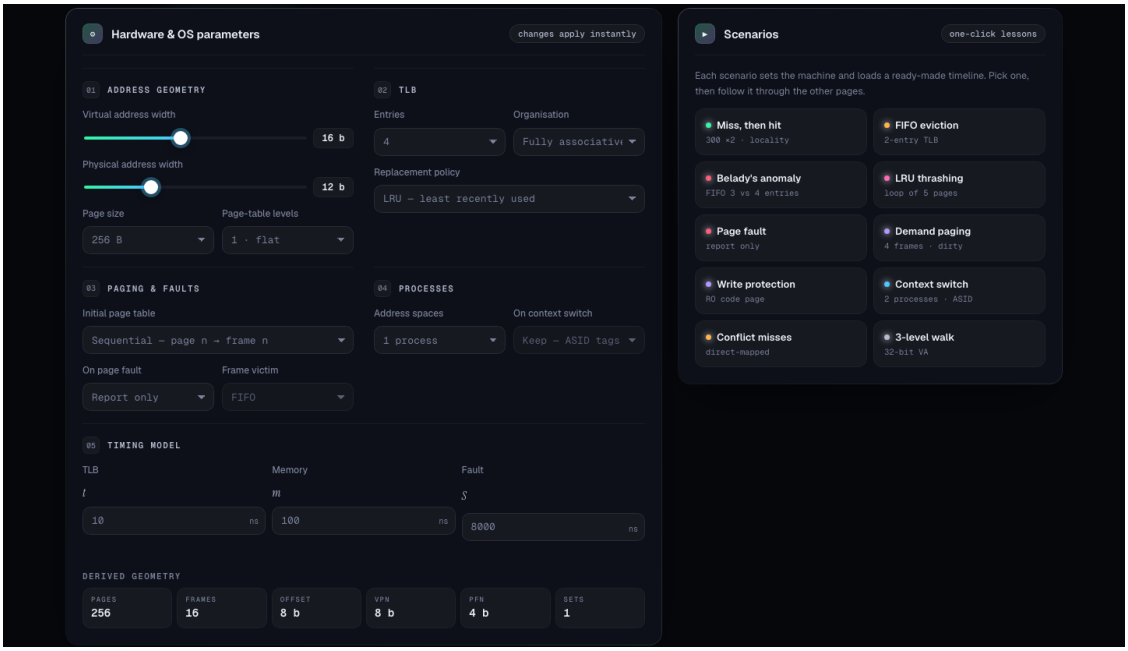



**Figure 8:** *Machine page: configuration, derived geometry and scenarios*

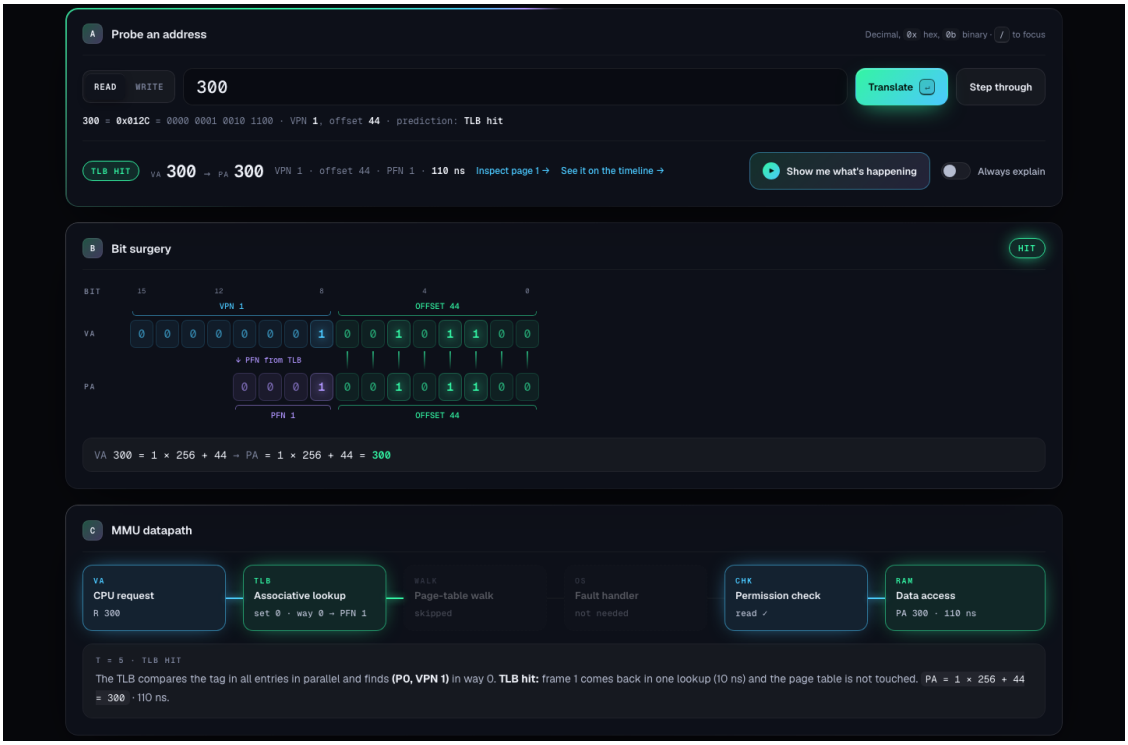



**Figure 9:** *Translate page: VA 300 is a TLB hit, PA 300 in 110 ns, with the bit-level view and datapath*

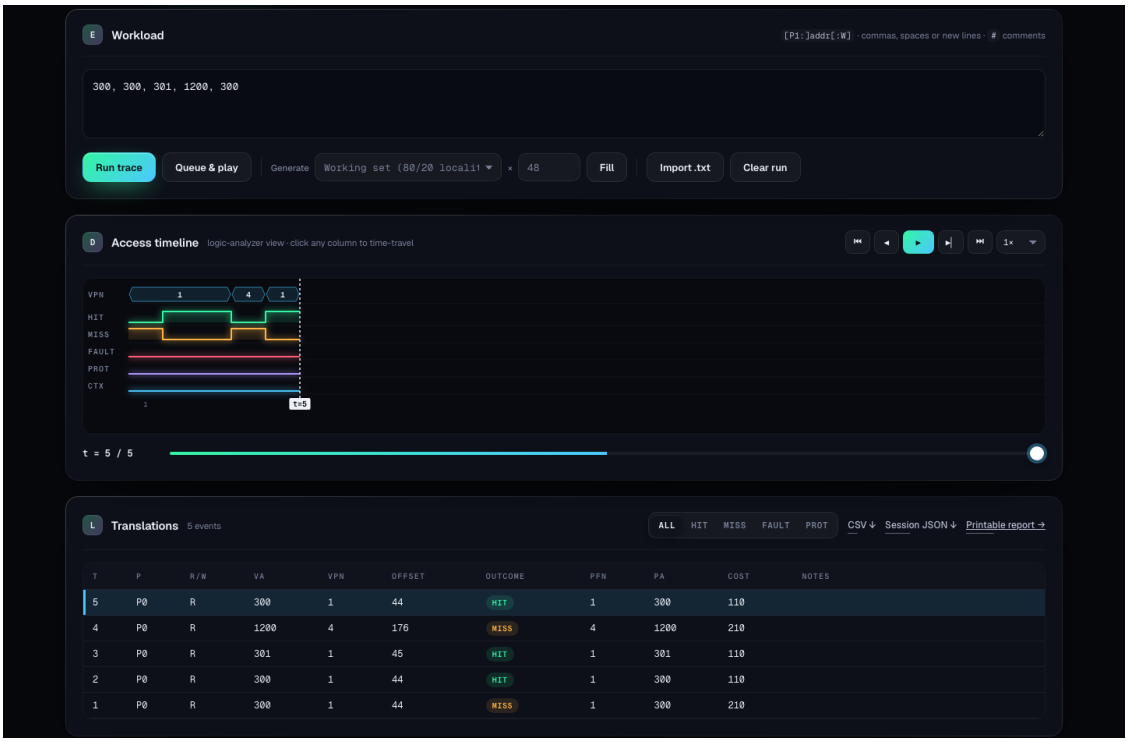



**Figure 10:** *Timeline page: trace, logic-analyzer lanes and event table*

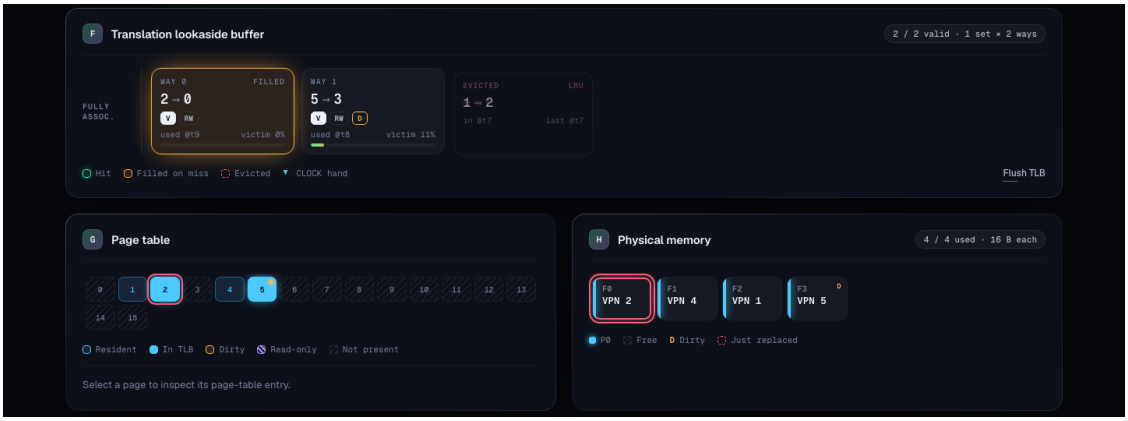



**Figure 11:** *Structures page: TLB ways, page-table map and physical frames*

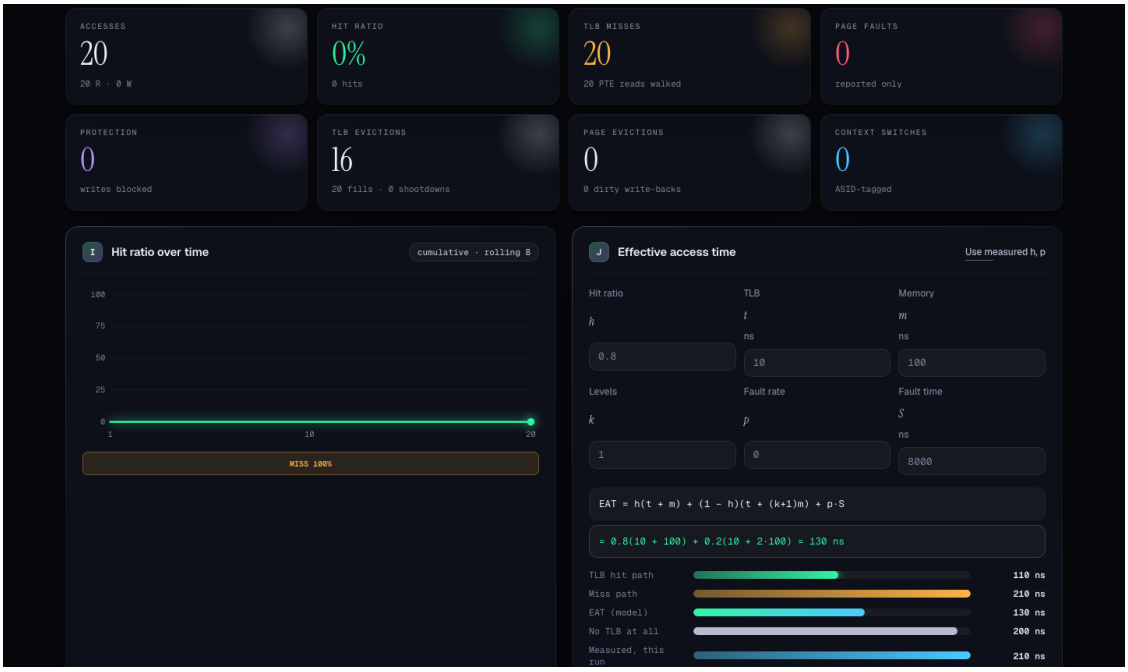



**Figure 12:** *Analytics page (LRU-thrashing scenario): 0 % hit ratio, and the EAT calculator: 130 ns with a TLB versus 200 ns without*

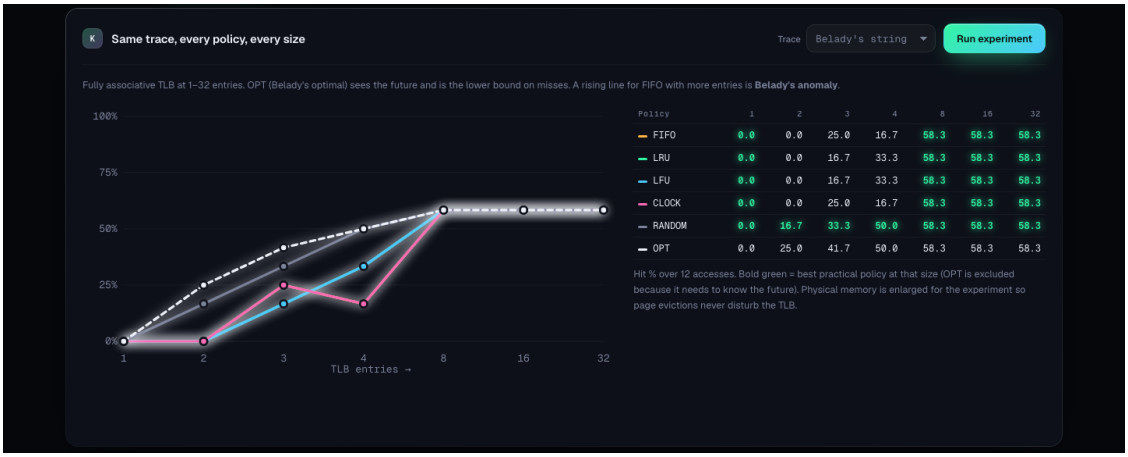



**Figure 13:** *Policy lab: hit ratio of six policies at seven TLB sizes on Belady's string*

## 6.3 Results

**Table 6:** *Results of the sample trace (default machine)*

| T | VA | VPN | Offset | Outcome | PA | Cost (ns) | Note |
|---|---|---|---|---|---|---|---|
| 1 | 300 | 1 | 44 | MISS | 300 | 210 | |
| 2 | 300 | 1 | 44 | HIT | 300 | 110 | |
| 3 | 301 | 1 | 45 | HIT | 301 | 110 | |
| 4 | 1200 | 4 | 176 | MISS | 1200 | 210 | |
| 5 | 6699 | 26 | 43 | FAULT | — | 110 | not serviced |
| 6 | 0 | 0 | 0 | MISS | 0 | 210 | |
| 7 | 256 | 1 | 0 | HIT | 256 | 110 | |
| 8 | 512 | 2 | 0 | MISS | 512 | 210 | |
| 9 | 768 | 3 | 0 | MISS | 768 | 210 | TLB evicts page 4 (LRU) |
| 10 | 0 | 0 | 0 | HIT | 0 | 110 | |
| 11 | 300 (W) | 1 | 44 | HIT | 300 | 110 | write: page 1 dirty |

**Table 7:** *Summary of the sample run*

| Measure | Value |
|---|---|
| Accesses | 11 (10 reads, 1 write) |
| TLB hits / misses / page faults | 5 / 6 / 1 |
| Hit ratio | 45.5 % |
| TLB evictions | 1 (page 4, chosen by LRU) |
| Measured average access time | (5 × 110 + 5 × 210 + 110) / 11 = 155.45 ns |
| Model EAT at h = 0.8 | 130 ns |
| Access time without a TLB | 200 ns |

**Table 8:** *Hit ratio (%) on Belady's string 1 2 3 4 1 2 5 1 2 3 4 5*

| Policy | 1 entry | 2 | 3 | 4 | 8 |
|---|---|---|---|---|---|
| FIFO | 0.0 | 0.0 | 25.0 | 16.7 | 58.3 |
| LRU | 0.0 | 0.0 | 16.7 | 33.3 | 58.3 |
| LFU | 0.0 | 0.0 | 16.7 | 33.3 | 58.3 |
| CLOCK | 0.0 | 0.0 | 25.0 | 16.7 | 58.3 |
| OPT | 0.0 | 25.0 | 41.7 | 50.0 | 58.3 |

## 6.4 Discussion of results

- **Locality pays off.** Repeated use of the same page (accesses 1 to 3) turns a 210 ns miss into 110 ns hits.
- **The TLB cuts the access time.** Even at the low hit ratio of this short trace (45.5 %), the average is 155.45 ns instead of 200 ns; at h = 0.8 the model gives 130 ns.

- **LRU chooses the right victim.** At T = 9 the TLB held pages 1, 4, 0 and 2; page 4 had not been used since T = 4, so LRU evicted it.
- **Belady's anomaly.** With FIFO (and CLOCK) the hit ratio fell from 25.0 % to 16.7 % when the TLB grew from 3 to 4 entries (9 misses became 10). LRU and OPT never got worse with more entries, and OPT was always the best.

# 7. Testing and Validation

## 7.1 Method

- **Unit tests with known answers.** Expected values were derived by hand or taken from textbook examples (such as Belady's 9 versus 10 misses and the EAT of 130 ns), not copied from the program's own output.
- **Two independent implementations.** The Python and JavaScript versions were run on the same inputs and their outputs compared.
- **Invariant checking.** After every access the JavaScript engine checks that hits + misses = accesses, that the TLB never exceeds its size or holds duplicates, and that every TLB entry agrees with a present page-table entry.
- **Mutation testing.** Bugs were planted on purpose (for example FIFO using the LRU key, or CLOCK ignoring reference bits) to confirm that the tests catch them.
- **Input validation.** Out-of-range and malformed addresses were entered to check that they are rejected with a clear message and without changing the state.

## 7.2 Python unit tests

All 16 Python tests pass (Figure 14).

**Table 9:** *Python unit tests*

| # | Test | Expected result | Result |
|---|---|---|---|
| 1 | Split VA 300 | VPN 1, offset 44, PFN 1, PA 300 | Pass |
| 2 | VA 300 twice | MISS then HIT; costs 210 and 110 ns | Pass |
| 3 | VA 4095 and 4096 | PA 4095; then FAULT with no PA | Pass |
| 4 | 300, 0x12C, 0b100101100, 0o454, 0x1_2C | All equal 300 | Pass |
| 5 | VA 65536 (out of range) | Rejected; state unchanged | Pass |
| 6 | FIFO, 3 entries, pages 1 2 3 1 4 1 | M M M H M M | Pass |
| 7 | LRU, 3 entries, pages 1 2 3 1 4 1 | M M M H M H | Pass |
| 8 | Belady's string, FIFO 3 and 4 entries | 9 and 10 misses | Pass |
| 9 | Belady's string, LRU and OPT | LRU 10 / 8, OPT 7 / 6 misses | Pass |
| 10 | CLOCK on 1 2 3 4 2 5 | Evicts page 3 (second chance for 2) | Pass |
| 11 | compare() on Belady's string | Anomaly reported for FIFO 3 → 4 | Pass |
| 12 | Direct-mapped TLB, pages 0 4 0 4 1 1 | Conflict misses: M M M M M H | Pass |
| 13 | Demand paging, 2 frames, pages 0(W) 1 2 | 3 faults, 1 eviction, 1 write-back | Pass |
| 14 | Write to read-only page | PROT, no PA, page stays clean | Pass |

| # | Test | Expected result | Result |
|---|---|---|---|
| 15 | EAT(h = 0.8, t = 10, m = 100) | 130 ns | Pass |
| 16 | Sample trace (11 accesses) | Hit ratio 5/11, average 1710/11 ns | Pass |

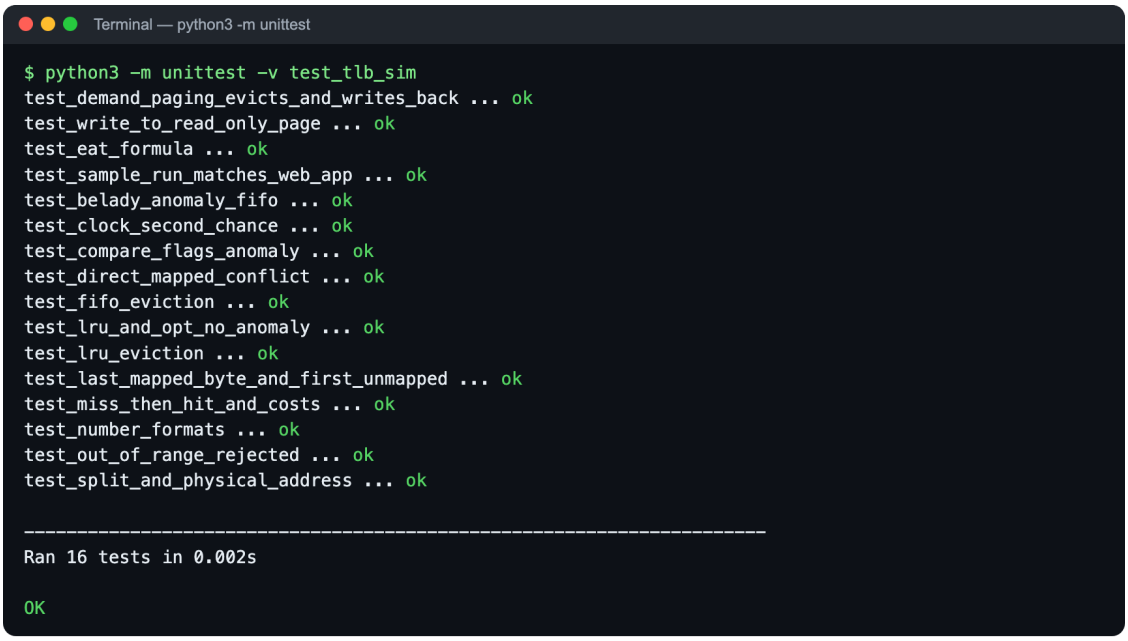



**Figure 14:** *Python unit tests: 16 of 16 pass*

### 7.3 JavaScript engine tests

The web engine has 96 automated tests (38 of them edge cases), run with `npm test` and on the Self-test page. All 96 pass (Figures 15 and 16).

**Table 10:** *JavaScript engine tests by group*

| Group | Passed |
|---|---|
| Configuration and geometry | 20 / 20 |
| Address and trace parsing | 11 / 11 |
| Translation arithmetic | 12 / 12 |
| TLB replacement policies | 19 / 19 |
| Page faults and demand paging | 10 / 10 |
| Protection and dirty bits | 5 / 5 |
| Processes, ASIDs and context switches | 5 / 5 |
| Statistics and effective access time | 8 / 8 |
| Workload generators and utilities | 6 / 6 |
| **Total** | **96 / 96** |

```
Terminal — npm test (JavaScript engine)

$ npm test
  ✓ EAT rejects h outside [0,1], negative times and non-numbers [edge]
  ✓ measured EAT of a run equals the analytic formula at the observed hit ratio
  ✓ all ratios are 0 (not NaN) before any access [edge]
  ✓ hits + misses = accesses and walks = misses on a mixed run
  ✓ k-level walk reads k page-table entries per miss
Workload generators & utilities
  ✓ every generator yields in-range, parseable accesses (default and tiny geometry) [edge]
  ✓ generators are deterministic for a given seed
  ✓ nextUseIndex points at the next access to the same page
  ✓ binary/hex helpers pad to the field width
  ✓ simulate() throws with the failing access index on bad input [edge]
  ✓ random initial mapping is reproducible and internally consistent
PASS  96/96 tests (38 edge cases) in 18.0 ms
Wrote TEST_RESULTS.md
```

**Figure 15:** *JavaScript engine tests: 96 of 96 pass*

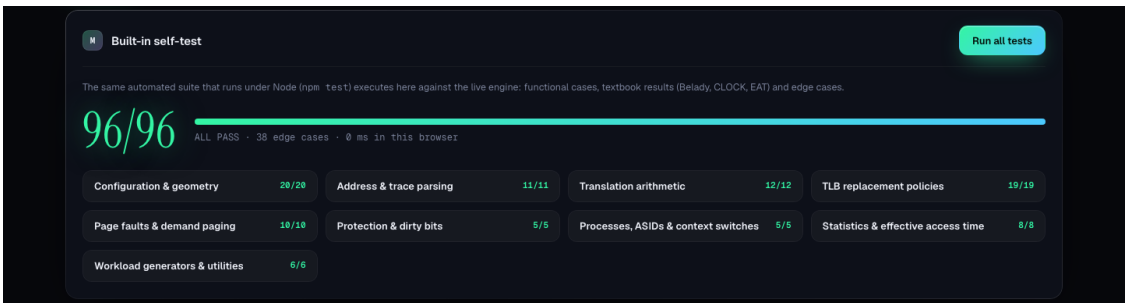



**Figure 16:** *Self-test page in the browser*

## 7.4 Cross-validation and input validation

**Table 11:** *Python versus web application on the same input*

| Check | Python | Web app | Match |
|---|---|---|---|
| Outcomes of the 11-access sample trace | MHHMFMHMMHH | MHHMFMHMMHH | Yes |
| Hit ratio | 45.5 % | 45.5 % | Yes |
| Measured average access time | 155.45 ns | 155.45 ns | Yes |
| TLB eviction at T = 9 | page 4 | page 4 | Yes |
| FIFO hit % at 3 / 4 entries | 25.0 / 16.7 | 25.0 / 16.7 | Yes |
| OPT hit % at 3 / 4 entries | 41.7 / 50.0 | 41.7 / 50.0 | Yes |

**Table 12:** *Invalid input handling*

| Input | Response | State changed? |
|---|---|---|
| Address 66990 (16-bit machine) | "address 66990 is outside the 16-bit virtual space (0–65,535)" | No |
| Address 65536 (Python) | ValueError: outside the 16-bit virtual space | No |
| Write to a read-only page | Protection fault; no physical address | Only the fault is counted |

# 8. Conclusion and Future Enhancements

## 8.1 Conclusion

The project met its objectives. The simulator translates any virtual address step by step, models the TLB with six replacement policies and three organisations, walks page tables of up to four levels, handles page faults and protection, and measures the hit ratio and effective access time. It was implemented in Python as a command-line program and in JavaScript as an interactive ten-page web application, and both give identical results.

The results show the value of the TLB clearly: a repeated access costs 110 ns instead of 210 ns, and at an 80 % hit ratio the average access time falls from 200 ns to 130 ns. The policy comparison reproduces Belady's anomaly for FIFO and confirms that LRU and OPT never get worse with a larger TLB. Correctness is supported by 16 Python tests and 96 JavaScript tests, all passing. By making an invisible hardware process visible and measurable, the project makes a hard Operating Systems topic easier to learn and to teach.

## 8.2 Future enhancements

- **Huge pages** (2 MiB and 1 GiB) with a separate large-page TLB.
- **Two-level TLB hierarchy** (L1 instruction and data TLBs with a shared L2 TLB).
- **Page-walk caches** and a more detailed timing model of the hardware walker.
- **Multi-core simulation** with TLB shootdowns by inter-processor interrupts.
- **Inverted and hashed page tables**, and working-set frame allocation.
- **Real address traces** captured from programs (for example with Valgrind) and replayed.
- **Nested paging** for virtual machines.
- **Quiz mode** with automatic grading for laboratory use.

# 9. References

[1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[2] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[5] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[6] A. Bhattacharjee and D. Lustig, *Architectural and Operating System Support for Virtual Memory*. Morgan & Claypool, 2017.

[7] Python Software Foundation, *The Python Standard Library* (dataclasses, unittest, argparse). Available: https://docs.python.org/3/library/

[8] Mozilla, MDN Web Docs (HTML, CSS and JavaScript). Available: https://developer.mozilla.org/

[9] Object Management Group, *Unified Modeling Language (UML) Specification*, version 2.5.1, 2017.